2. Gyakorlat

Python kurzus

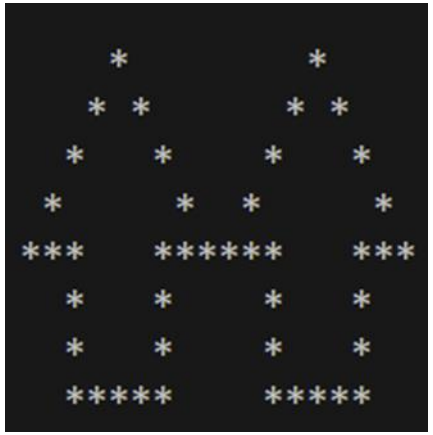
Adattípusok és változók
Vezérlési szerkezetek

print(object*(s)*, sep=*separator*, end=*end*, …)

```
print("I'm", "learning", "Python", sep = "---", end = "****")
print()   vagy   print('\n')
print("I'm", "learning", "Python", sep = "\n")

print('22'+'333')
print(4*('22'+'333'))
print('\"I\'m\"') → "I'm"

print("+"+ 10*"-"+"+")
print(("|"+ " "*10 + "|\n")*5,end="")
print("+" + 10 * "-"+"+")
print()
```



input(*prompt*)

```
x = input('Kérek egy egész számot: ')
y = int(input('Kérek egy egész számot: '))
```

További **print** formázási lehetőségek, példák:

1. print ("**%s** széles: **%d**, hosszú: **%d**, magas: **%f**" **%** ('Téglatest adatai:',12 ,35, 14.6))

   Téglatest adatai: szélesség: 12, hosszúság: 35, magasság: 14.600000

   a, b, c = 12, 35, 14.6
   print ("Széles: %d, hosszú: %d, magas: %f" % (a, b, c))

2. print("Széles: {}, hosszú: {}, magas: {}".**format**(a, b, c))

   print("A téglatest térfogata =", a*b*c, "egység")

   Széles: 12, hosszú: 35, magas: 14.6
   A téglatest térfogata = 6132.0 egység

   print("cím: {}/{}, idő: {}, tartam: {:.2f}".**format**("Delta", 12, '16:45', 46.278))

   cím: Delta/12, idő: 16:45, időtartam: 46.28 perc

3. márka = "Opel"
   típus = "Astra"
   print**(f"**Az {típus} egy {márka} gyártmány.**")**

   Az Astra egy Opel gyártmány.

A Python objektumok:

- Pl. szám, string, adatstruktúra, függvény, osztály, modul
- Minden objektumnak van **neve** és:
  - **belső adata:** pl. **literál**ok: 1, 3.5, ”a”, False, . . .
  - **adattípusa** (pl. integer, string, boolean, list, . . . , class)
  - **műveletei** és **metódusai**

Példa:

```
a = [0,1,2,3]    b= [4,5,6,8]

c = a + b        → [0,1,2,3,4,5,6,8]
d = 3*[0,1,2]    → [0,1,2,0,1,2,0,1,2]
b.append(9)      → [4,5,6,8,9]
```

Változónevek

- Kifejezőek: **`össz_1, Hossz, FirstName, x0, y_0, age, ...`**
- Szabályok:
  - betűvel vagy aláhúzásjellel kell kezdődnie **`_time_ , kép2`**
  - <u>nem kezdődhet számmal</u> pl. **`1_sor`** nem lehet
  - csak alfanumerikus karakterek és aláhúzásjelek pl. **`n!`** nem lehet
  - kis- és nagybetűket megkülönbözteti pl. **`ALma <> Alma`**
- A kulcsszavak tiltottak:

and	assert	break	class	continue	def
del	elif	else	except	exec	finally
for	from	global	if	import	in
is	lambda	not	or	pass	print
raise	return	try	with	while	yield
False	None	True			

Alapvető adattípusok

- **Elemi adattípusok:**
  - Egész szám (integer) — 12 — int()
  - Valós szám (float) — 12.34 — float()
  - Komplex szám (complex) — 1.2 + 3.4j — complex()
  - Logikai (boolean) — True v. False — bool()
- **Összetett adattípusok:**
  - Szöveg (string) — ”Almafa” v. ’Almafa’ — str()
  - Tuple — (10, 20, 30, 40) — tuple()
  - Lista (list) — [1, 2, 2, 3, 3, 3, 4, 4, 4, 4] — list()
  - Szótár (dictionary) — {a: 1, b: 2, c: 3, d: 4} — dict()
  - Halmaz (set) — {1, 2, 3, 4} — set()

Változónevek:

Nm_3 2_szám _hetek h123”4 False false

Adat típusok:

1234 ”1010” 3.14 ”feladat” True 1,2,3,4

(”lap”, ”oldal”, ”szám”) [x, y, z, x, ”z”, 1, ’1’] {x, y, z}

Mi az output?

print(0.000000000002)

print(0o123)

print(0x123)

print(2) és print(’2’)

Értékadások

Egyszerű értékadás:

változó = érték

```
alma = 26
```

Többszörös (szimultán) értékadás:

$\text{változó}_1, \ldots, \text{változó}_n = \text{érték}_1, \ldots, \text{érték}_n$

```
alma, körte, dió = 26, 15, 4
alma = körte = dió = 15
```

Műveletek és operátorok

Aritmetikai műveletek:

Symbol	Task Performed	Example	Result
+	Addition	4 + 3	7
-	Subtraction	4 - 3	1
/	Division	7 / 2	3.5
%	Mod	7 % 2	1
*	Multiplication	4 * 3	12
//	Floor division	7 // 2	3
**	Power of	7 ** 2	49

Figyelembe kell venni: prioritások, nullával osztás elkerülése

Összehasonlító operátorok: ==, !=, <, >, <=, >=

Prioritás	Operátor	neve	
1	+, -	előjel	egyváltozós
2	**	hatványozás	kétváltozós
3	*, /, //, %	szorzás, …	kétváltozós
4	+, -	összeadás, kivonás	kétváltozós

Példa:

```
a = 5
b = 12
c = (a**2 + b**2)**0.5
print(”c = ”, c)
```

Mennyi az eredménye:

3 + 4*2 = ?	9%5%3 = ?	2**3**2 = ?	5**4**0.5 = ?
3*4%5 = ?	16%5/5*2 = ?	4.5 // 2 = ?	4.5 // 2.5 = ?
0 % 5 = ?	5 % 0 = ?	2 // 4 = ?	- 2 // 4 = ?

```
print(2 ** 3)        print(2 ** 3.)
print(2. ** 3)       print(2. ** 3.)

print(3 * 4)         print(3 * 4.)
print(3. * 4)        print(3. * 4.)

print(12 / 3)        print(12 / 3.)
print(12. / 3)       print(12. / 3.)

print(-6 // 2)       print(6 // 2.)
print(6. // 2)       print(6. // 2.)

print(13 % 3)        print(12 % 4.2)

print(-4 + 4)        print(-4. + 8)
print(-4 - 4)        print(4. - 8)
```

```
var = 2
var = 3
print(var)

a = '2'
b = "3"
print(a + b)
```

Shortcut:

```
a = 6
b = 3
a /= 2 * b
print(a)
```

Logikai műveletek:

Logikai értékek: True, False
Logikai műveletek: not, and, or, xor (^)

```
x, y = 1, 0
z = ((x == y) and (x != y)) or not(x == y)
print(not(z))
```

Bitenkénti logikai műveletek: ~, &, |, ^, >>, <<

A	B	~B	A&B	A|B	A^B
1	1	0	1	1	0
1	0	1	0	1	1
0	1	0	0	1	1
0	0	1	0	0	0

```
print(0^0)
print(0^1)
print(1^0)
print(1^1)
print(8^2)
print(8^10)
```

```
print(0|0)
print(0|1)
print(1|0)
print(1|1)
print(7|2)
print(7|8)
```

```
print(bin(7))
print(0b011)
```

```
0b111
3
```

```
x >> 1        # Osztunk 2-vel
x << 2        # Szorzunk 4-gyel
```

```
x = 4                    x = 0000 0100
y = 1                    y = 0000 0001

a = x & y                a = 0000 0000
b = x | y                b = 0000 0101
c = ~x                   c = 1111 1011
d = x ^ 5                d = 0000 0001
e = x >> 2               e = 0000 0001
f = x << 2               f = 0001 0000

print(a, b, c, d, e, f)
```

```
0 5 -5 1 1 16
```

1. feladat: Időpont kiszámítása

Kérd be egy esemény kezdő időpontját óra és perc megadásával. Kérj be egy időtartamot percben. Írd ki az esemény befejezésének időpontját a minta szerinti formában:

A bemenet:
13
20
45
Az elvárt kimenet:
14:05

Megj.

- Időpontok 00:00 órától 23:59-ig
- Oldd meg az éjfél átlépést is helyesen:

A bemenet:
23
50
45
Az elvárt kimenet:
0:35

Házi feladat:

Kérj be két időpontot óó pp mm formában.

a) Határozd meg a két időpont távolságát másodpercben, majd ezt az időtartamot írd ki óó:pp:mm formában.

b) Határozd meg azt az újabb időpontot, amelyik a későbbitől ugyanolyan messze van, mint a későbbi a korábbitól!
Ennek a kimeneti formája: óó:pp:mm legyen.

```
Kérem az első időpontot (óra perc másodperc): 12 34 10
Kérem a második időpontot (óra perc másodperc): 15 36 14
Az időpontok közötti különbség: 3:2:4
Az új időpont: 18:38:18
```

Feltételes utasítások

```
x = 10
if x == 10:
    print(x == 10)
if x > 5:
    print(x > 5)
if x < 10:
    print(x < 10)
else:
    print("else")
```

```
True
True
else
```

```
x = 10
if x != 10:
    print(x != 10)
elif x < 5:
    print(x < 5)
elif x > 10:
    print(x > 10)
else:
    print("else")
```

```
else
```

```
x = 10
if x != 10:
    print(x != 10)
elif x > 5:
    print("x > 5")
elif x == 10:
    print(x == 10)
else:
    print("else")
```

```
x > 5
```

2. feladat: Szökőév

A normál évek 365 napból állnak, a szökőévek pedig 366 napból.
A Gergely-naptár bevezetésétől (1582) a következő szabállyal
határozzuk meg az év típusát:

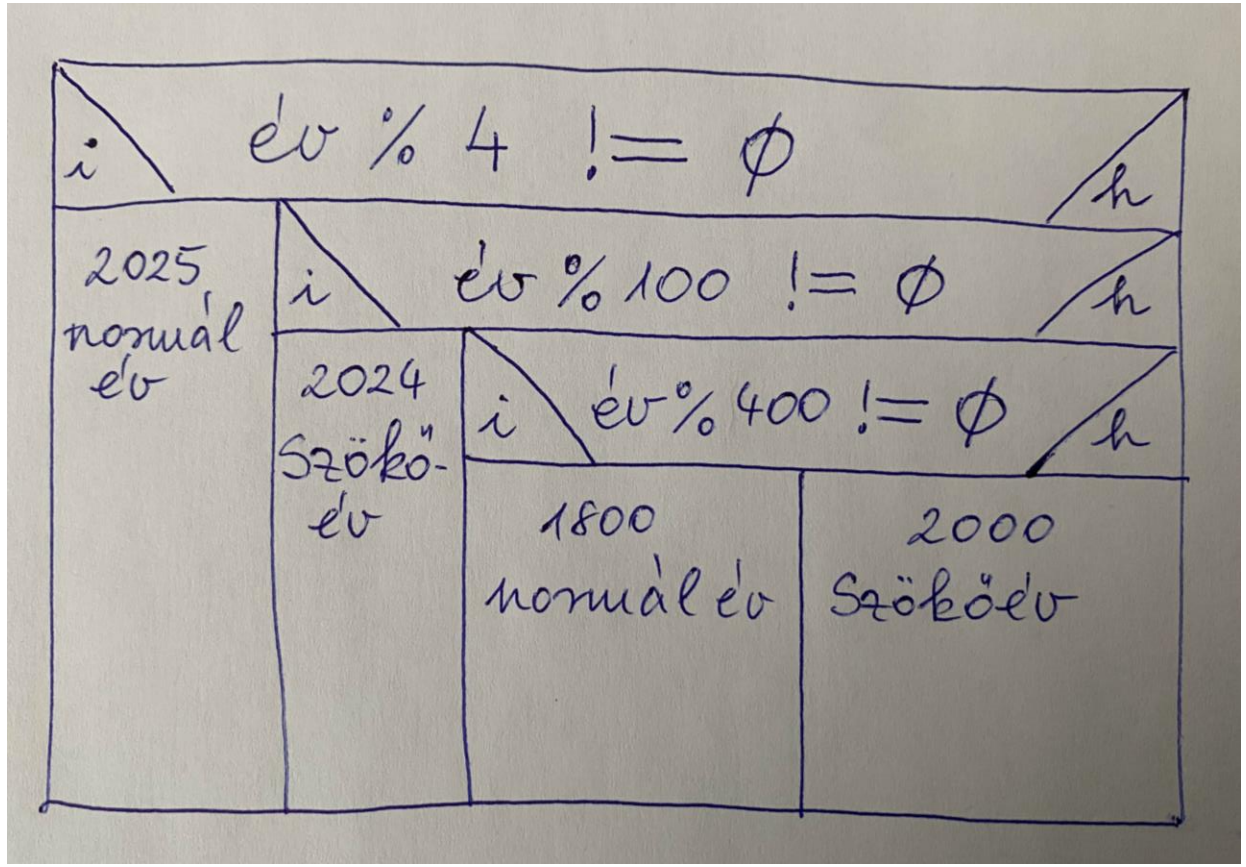
- ha az évszám nem osztható néggyel, akkor az normál év;
- egyébként, ha az évszám nem osztható 100-zal, akkor szökőév;
- egyébként, ha az évszám nem osztható 400-zal, akkor normál év;
- egyébként szökőév.

Olvass be egy évszámot. Írd ki, hogy normál év vagy szökőév.
Ellenőrizd, hogy a beírt év a Gergely-naptári időszakba esik-e,
különben adj egy figyelmeztetést:
Nem a Gergely-naptári időszakon belül van.

Használd a `!=` és `%` operátorokat.
Teszteld a kódot a megadott adatok alapján.

```
Kérem az évet: 2024
Szökőév
Kérem az évet: 2025
Normál év
Kérem az évet: 2000
Szökőév
Kérem az évet: 1999
Normál év
```

```
Kérem az évet: 1200
Nem a Gergely-naptári időszakon belül van.
```


i
év % 4 != ϕ
h
2025
normál
év
i
év % 100 != ϕ
h
2024
Szökő-
év
i
év % 400 != ϕ
h
1800
normál év
2000
Szökőév

A while ciklus

- A while ciklus:

```
while feltétel:
    utasítás_1
    utasítás_2
    .
    .
    utasítás_n
```

```
i = 0                          #kezdőérték megadása
while i < 5:                   #ciklusfeltétel megadása
    print(i)                   #ciklusmag
    i += 1                     #léptetés
print("Ciklus után", i)
```

```
0
1
2
3
4
Ciklus után 5
```

- Végtelen ciklus:

```
while True:
    print("Benn ragadtam egy ciklusban.")
```

Interrupt a billentyűzetről: Ctrl + c

3. feladat: Collatz sejtés

1937-ben Lothar **Collatz** német matematikus egy érdekes sejtést fogalmazott meg:

1. vegyünk egy c0 tetszőleges pozitív egész számot;
2. ha páros, akkor legyen az új c0 = c0 : 2;
3. ha páratlan, akkor legyen új c0 = 3 * c0 + 1;
4. ha c0 ≠ 1, ugorjunk a 2. pontra.

A sejtés szerint a c0 kezdeti értékétől függetlenül mindig elérjük az 1-et.

Írj programot, amely beolvas egy pozitív egész számot, és végrehajtja a fenti lépéseket, amíg c0 különbözik 1-től. Számold meg a cél eléréséhez szükséges lépéseket, és írd ki a c0 összes köztes értékét is.

Használj while ciklust.
Teszteld a kódot a megadott adatok alapján.

```
Kérek egy egész számot! 16
1. lépés: c0 = 8
2. lépés: c0 = 4
3. lépés: c0 = 2
4. lépés: c0 = 1
```

```
Kérek egy egész számot! 15
1. lépés: c0 = 46
2. lépés: c0 = 23
3. lépés: c0 = 70
4. lépés: c0 = 35
5. lépés: c0 = 106
6. lépés: c0 = 53
7. lépés: c0 = 160
8. lépés: c0 = 80
9. lépés: c0 = 40
10. lépés: c0 = 20
11. lépés: c0 = 10
12. lépés: c0 = 5
13. lépés: c0 = 16
14. lépés: c0 = 8
15. lépés: c0 = 4
16. lépés: c0 = 2
17. lépés: c0 = 1
```

Házi feladat: Címletekre bontás

Írj programot, amely beolvas egy pénzösszeget, majd ezt a **lehető legnagyobb** címletekre bontja. A címletek legyenek: 20, 10, 5, 2 és 1 ezer Ft, valamint 500, 200, 100, 50, 20, 10 és 5 Ft.
Írasd ki, hogy melyik címletből mennyit és milyen összegben használt fel.
Használd a **lista.pop() metódust** a címletek lista utolsó elemének eltávolításához.

```
Kérem az összeget (Ft): 125475
Címletezve:
6 db 20000 Ft = 120000 Ft
1 db 5000 Ft = 5000 Ft
2 db 200 Ft = 400 Ft
1 db 50 Ft = 50 Ft
1 db 20 Ft = 20 Ft
1 db 5 Ft = 5 Ft
```

Használj while ciklust.
Teszteld a kódot a megadott adatok alapján.

A for ciklus:

```
for i in range(2,10,3):
   print(i)
```

```
2
5
8
```

```
for i in range(10,2,-3):
   print(i)
```

```
10
 7
 4
```

```
for i in range(5):
   print(i, end=' ')
   print(i*'#')
print("A ciklus után", i)
```

```
0
1 #
2 ##
3 ###
4 ####
A ciklus után 4
```

```
#break példa

for i in range(1, 6):
   if i == 3:
      break
   print("A cikluson belül:", i)
print("A cikluson kívül.")
```

```
A cikluson belül: 1
A cikluson belül: 2
A cikluson kívül.
```

```
#continue példa

for i in range(1, 6):
   if i == 3:
      continue
   print("A cikluson belül:", i)
print("A cikluson kívül.")
```

```
A cikluson belül: 1
A cikluson belül: 2
A cikluson belül: 4
A cikluson belül: 5
A cikluson kívül.
```

4. feladat: Magánhangzó-evő

Írj egy programot, amely használ:

- for ciklust
- feltételes utasítást (if-elif-else)

Kérd meg a felhasználót, hogy írjon be egy szót.

Használd a **szó = szó.upper() metódust** a szó nagybetűssé alakításához.

A bevitt szóból válogasd ki a mássalhangzókat, használd az összefűzési műveletet és a kapott **csak_mással** változót írd ki a képernyőre.

Kérem a szót:
Üvegtigris
VGTGRS

Kérem a szót:
kávéfőző
KVFZ

Házi feladat: Szómanipuláló

Kérj be egy szót. (Az angol ABC betűit használd a feladatban.)

1. Válogasd szét a magánhangzóit és a mássalhangzóit két csoportba, és írd ki ezeket.
2. Készíts egy tükörszót, vagyis az eredeti szóhoz ragaszd hozzá a megfordított szót. Pl. szó = „kabala” → új_szó = „kabalaalabak”
3. Rendezd ABC rendbe a szó betűit, és írd ki a kapott stringet.
4. Titkosítsd a szót úgy, hogy a betűit rendre a betűrendben utána következő betűvel helyettesíted, pl. „alma” → „bmnb”
   A „z” betű helyett az „a” következzen, pl. „azaz” → „baba”

   (Caesar-rejtjelezés)

Köszönöm a figyelmet!

Konzultáció

Minden héten csütörtökön 18:00 – 20:00